RF Cross validation

0.Packages

```
## This note book take 15-18 mins to run, be caseful lol.
R_packages <- c("readr","dplyr","lubridate",
                "randomForest","quantregForest","BART", "tidyr", "ranger", "knitr")
options(repos=c(CRAN="http://cran.rstudio.com"))

## Use Burda style package
if(!requireNamespace("librarian", quietly=TRUE)) install.packages("librarian")
librarian::shelf(R_packages)
```

```
## Warning: package 'readr' was built under R version 4.5.2
```

1. Data

```
# copy from VAR, same thing
df_ml <- read_csv("balanced_can_md.csv", show_col_types = FALSE) %>%
  mutate(
    Date      = as.Date(Date),
    GDP_per_w = GDP_new - EMP_CAN
  ) %>%
  arrange(Date)

targets <- c("GDP_per_w", "CPI_ALL_CAN", "NHOUSE_P_CAN")
features_large <- c(
  "GDP_per_w", "CPI_ALL_CAN", "NHOUSE_P_CAN", "IP_new",
  "BSI_new", "UNEMP_CAN", "EMP_CONS_CAN", "hstart_CAN_new",
  "CRED_HOUS", "CRED_HOUS_MORT", "BANK_RATE_L", "TBILL_3M",
  "Exp_BP_new", "Imp_BP_new", "IPPI_METAL_CAN", "OILP_new",
  "WTISPLC"
)

setdiff(features_large, names(df_ml))
```

```
## character(0)
```

2. HELPER: build laged panels

```r
build_lagged_panel <- function(df, predictors, targets, p, h) {
  dat <- df %>%
    dplyr::select(Date, dplyr::all_of(unique(c(predictors, targets)))) %>%
    arrange(Date)

  for (lag in 1:p) {
    for (v in predictors) {
      new_name <- paste0("L", lag, "_", v)
      dat[[new_name]] <- dplyr::lag(dat[[v]], lag)
    }
  }

  for (v in targets) {
    lead_name <- paste0("F", h, "_", v)
    dat[[lead_name]] <- dplyr::lead(dat[[v]], h)
  }

  lag_cols <- grep("^L[0-9]+_", names(dat), value = TRUE)
  y_cols   <- paste0("F", h, "_", targets)

  dat_final <- dat %>%
    dplyr::select(dplyr::all_of(c(lag_cols, y_cols))) %>%
    tidyr::drop_na()

  list(
    X = as.matrix(dat_final[, lag_cols, drop = FALSE]),
    Y = as.matrix(dat_final[, y_cols,  drop = FALSE])  # columns in same order as targets
  )
}
```

3a. Time Series K-fold CV

```r
rf_ts_cv_rmse_fast <- function(df, predictors, targets,
                               p,
                               K         = 10,
                               max_h,
                               num.trees = 200,
                               mtry_frac = 1/3) {

  se_all <- list()

  for (h in 1:max_h) {
    panel <- build_lagged_panel(df, predictors, targets, p, h)
    X <- panel$X
    Y <- panel$Y
    n <- nrow(X)
    n_y <- ncol(Y)   # 3 targets

    if (n < K + 5) stop("Too few obs after lags/leads for this p / h.")
```

```
    start_idx <- floor(0.6 * n)
    fold_ends <- floor(seq(from = start_idx, to = n - 1, length.out = K))

    se_h <- matrix(NA_real_, nrow = 0, ncol = n_y)

    for (end_idx in fold_ends) {
      train_idx <- 1:end_idx
      test_idx  <- end_idx + 1
      if (test_idx > n) break

      x_train <- X[train_idx, , drop = FALSE]
      y_train <- Y[train_idx, , drop = FALSE]
      x_test  <- X[test_idx, , drop = FALSE]

      mtry_val <- max(1L, floor(ncol(x_train) * mtry_frac))

      preds <- numeric(n_y)


      for (j in seq_len(n_y)) {
        rf_fit <- ranger::ranger(
          dependent.variable.name = "y",
          data       = data.frame(y = y_train[, j], x_train),
          num.trees  = num.trees,
          mtry       = mtry_val,
          write.forest = TRUE,
          respect.unordered.factors = "order"
        )

        preds[j] <- predict(rf_fit, data = as.data.frame(x_test))$predictions
      }


      se_h <- rbind(se_h, (Y[test_idx, ] - preds)^2)
    }

    se_all[[h]] <- se_h
  }

  se_cat <- do.call(rbind, se_all)
  sqrt(mean(se_cat, na.rm = TRUE))
}
```

3B. Helper: search over p with cheap RF (100)

```
rf_search_p <- function(df, predictors, targets,
                        p_grid,
                        K,
                        max_h,
                        num.trees_search = 100) {
```

```
  res <- data.frame(p = integer(), RMSE = numeric())

  for (p in p_grid) {
    rmse_p <- rf_ts_cv_rmse_fast(
      df         = df,
      predictors = predictors,
      targets    = targets,
      p          = p,
      K          = K,
      max_h      = max_h,
      num.trees  = num.trees_search
    )

    res <- rbind(res, data.frame(p = p, RMSE = rmse_p))
    cat("RF search: p =", p, "RMSE =", round(rmse_p, 6),
        " (max_h =", max_h, ")\n")
  }

  res
}
```

4A. Short horizon

```
## it take 5 minutes
p_grid <- 1:12  # Victor your wish: check all lags

results_rf_short_search <- rf_search_p(
  df         = df_ml,
  predictors = features_large,
  targets    = targets,
  p_grid     = p_grid,
  K          = 10,
  max_h      = 3,
  num.trees_search = 100
)
```

```
## RF search: p = 1 RMSE = 0.002478  (max_h = 3 )
## RF search: p = 2 RMSE = 0.00252  (max_h = 3 )
## RF search: p = 3 RMSE = 0.002571  (max_h = 3 )
## RF search: p = 4 RMSE = 0.002675  (max_h = 3 )
## RF search: p = 5 RMSE = 0.00256  (max_h = 3 )
## RF search: p = 6 RMSE = 0.002386  (max_h = 3 )
## RF search: p = 7 RMSE = 0.002423  (max_h = 3 )
## RF search: p = 8 RMSE = 0.002584  (max_h = 3 )
## RF search: p = 9 RMSE = 0.002651  (max_h = 3 )
## RF search: p = 10 RMSE = 0.002589  (max_h = 3 )
## RF search: p = 11 RMSE = 0.002383  (max_h = 3 )
## RF search: p = 12 RMSE = 0.002369  (max_h = 3 )
```

```
results_rf_short_search
```

```
##     p        RMSE
## 1   1 0.002477795
## 2   2 0.002520004
## 3   3 0.002570891
## 4   4 0.002674743
## 5   5 0.002560046
## 6   6 0.002385729
## 7   7 0.002422888
## 8   8 0.002584346
## 9   9 0.002651317
## 10 10 0.002589188
## 11 11 0.002382692
## 12 12 0.002369480
```

```
best_row_short <- results_rf_short_search[
  which.min(results_rf_short_search$RMSE), ]
best_p_rf_short <- best_row_short$p
best_row_short
```

```
##     p       RMSE
## 12 12 0.00236948
```

```
rmse_rf_short_final <- rf_ts_cv_rmse_fast(
  df         = df_ml,
  predictors = features_large,
  targets    = targets,
  p          = best_p_rf_short,
  K          = 10,
  max_h      = 3,
  num.trees  = 500  # fancy lol
)

short_term_rf_table <- data.frame(
  Model      = "RF (17 variables)",
  Lag_p      = best_p_rf_short,
  RMSE_1to3  = rmse_rf_short_final
)

short_term_rf_table
```

```
##               Model Lag_p   RMSE_1to3
## 1 RF (17 variables)    12 0.002315975
```

4B. Long Dimension

```
## it take 15 minutes, no joke
results_rf_long_search <- rf_search_p(
  df         = df_ml,
```

```
  predictors = features_large,
  targets    = targets,
  p_grid     = p_grid,
  K          = 10,
  max_h      = 12,
  num.trees_search = 100
)
```

```
## RF search: p = 1 RMSE = 0.002638  (max_h = 12 )
## RF search: p = 2 RMSE = 0.002624  (max_h = 12 )
## RF search: p = 3 RMSE = 0.002576  (max_h = 12 )
## RF search: p = 4 RMSE = 0.002548  (max_h = 12 )
## RF search: p = 5 RMSE = 0.00257  (max_h = 12 )
## RF search: p = 6 RMSE = 0.002629  (max_h = 12 )
## RF search: p = 7 RMSE = 0.002583  (max_h = 12 )
## RF search: p = 8 RMSE = 0.002637  (max_h = 12 )
## RF search: p = 9 RMSE = 0.002672  (max_h = 12 )
## RF search: p = 10 RMSE = 0.002668  (max_h = 12 )
## RF search: p = 11 RMSE = 0.002682  (max_h = 12 )
## RF search: p = 12 RMSE = 0.002675  (max_h = 12 )
```

```
results_rf_long_search
```

```
##     p        RMSE
## 1   1 0.002637655
## 2   2 0.002623541
## 3   3 0.002575677
## 4   4 0.002548100
## 5   5 0.002570385
## 6   6 0.002628597
## 7   7 0.002582974
## 8   8 0.002636632
## 9   9 0.002672174
## 10 10 0.002667640
## 11 11 0.002682279
## 12 12 0.002674783
```

```
best_row_long <- results_rf_long_search[
  which.min(results_rf_long_search$RMSE), ]
best_p_rf_long <- best_row_long$p
best_row_long
```

```
##   p      RMSE
## 4 4 0.0025481
```

```
rmse_rf_long_final <- rf_ts_cv_rmse_fast(
  df         = df_ml,
  predictors = features_large,
  targets    = targets,
  p          = best_p_rf_long,
  K          = 10,
```

```
  max_h      = 12,
  num.trees  = 500
)

long_term_rf_table <- data.frame(
  Model       = "RF (17 variables)",
  Lag_p       = best_p_rf_long,
  RMSE_1to12  = rmse_rf_long_final
)

long_term_rf_table
```

```
##                Model Lag_p  RMSE_1to12
## 1 RF (17 variables)     4 0.002562844
```